

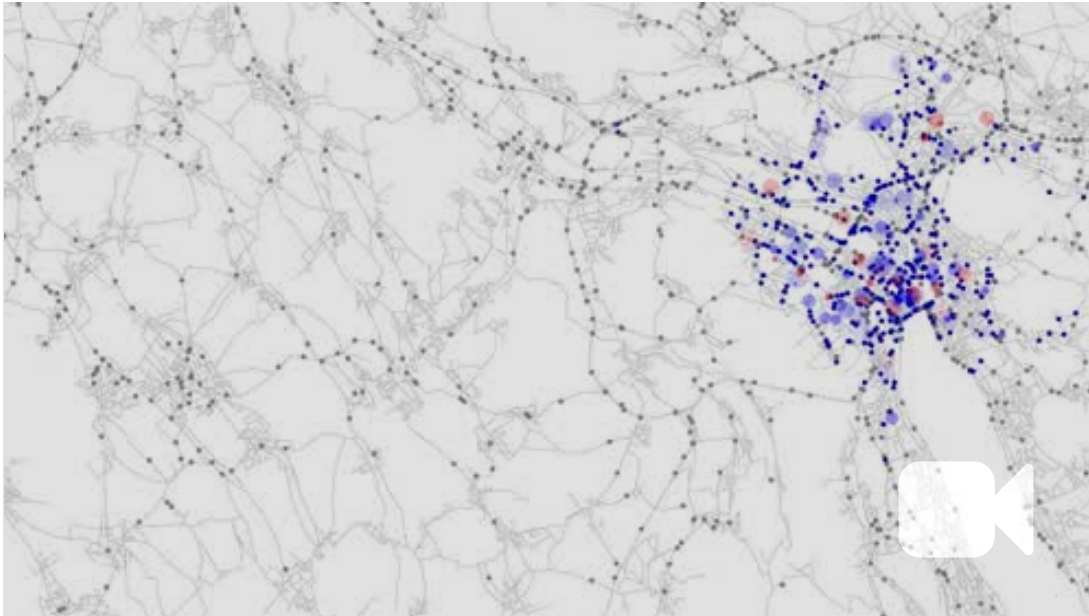
Benchmarking on-demand mobility algorithms using a new remote dispatching interface for MATSim

Sebastian Hörl

28 September 2026

MATSim User Meeting 2026

- MATSim provides the DRT / DVRP packages that allow simulating on-demand mobility services
- As standalone simulations (given fixed trips) or as part of the whole loop (agent deciding for DRT)

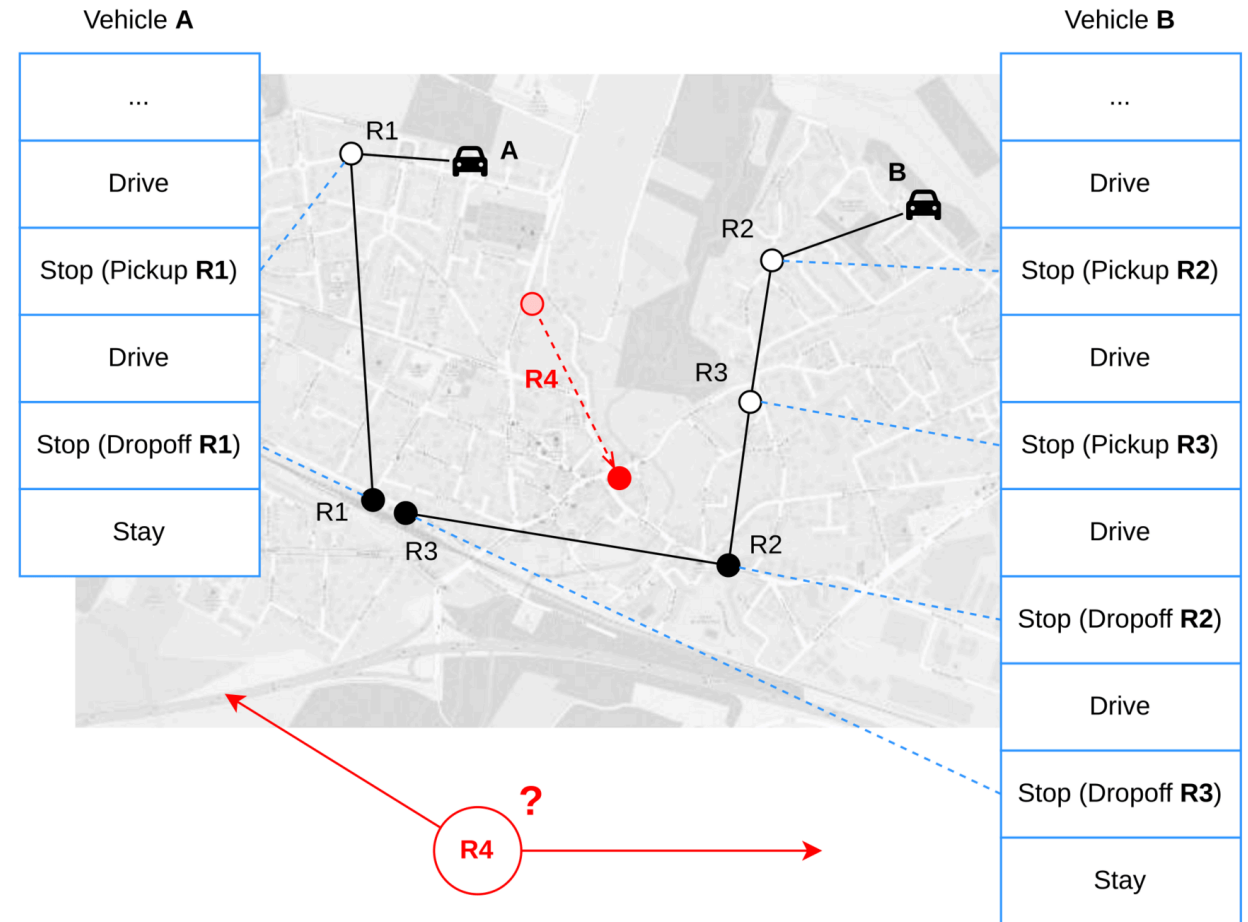


Fleet dispatching

- Fleet dispatching means defining which vehicle should pick up and drop off which requests and in which order
- Assignments need to adhere to constraints (latest pickup time, latest dropoff time, maximum detour, ...)
- **DRT Algorithm**
Default implementation in MATSim
Myopic insertion heuristic
- **Alonso-Mora**
Overrides default implementation
Dynamic reconstruction of schedules



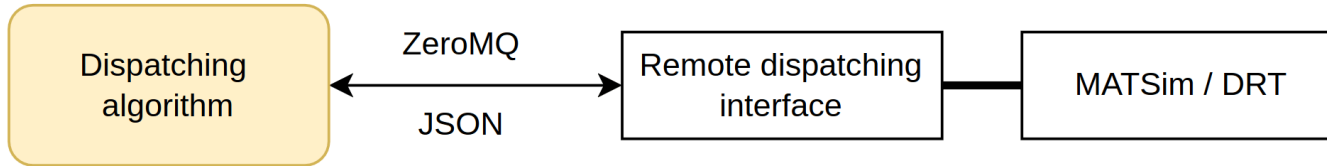
[matsim-org/alonso-mora](https://github.com/matsim-org/alonso-mora)



- Let's flexibilize the way we can hook dispatching algorithms into MATSim!
- Rapid development using Python and other languages
- Reuse of existing libraries (machine learning, inference, ...)
- Reuse of existing algorithms in literature
- Concrete use case: connecting Fleetpy to MATSim



TUM-VT/FleetPy



- MATSim (server) and dispatcher (client)
- Connection via ZeroMQ
 - REQREP mode: one request follows a response follows a request
 - Keeping the message exchange simple
- Messages encoded in JSON
 - Not optimal, but easy to follow (potential to replace)
- Dispatching step parametrizable in MATSim, for each
 - MATSim sends **state** (vehicles, requests, ...)
 - Dispatcher answer with **assignment** (vehicle schedules, ...)

```
{
  "@message": "state",
  "time": 7200.0,
  "pickups": [
    { "request": "req1", "vehicle": "veh1", "ongoing": true },
    { "request": "req2", "vehicle": "veh62", "ongoing": false }
  ],
  "dropoffs": [
    { "request": "req5", "vehicle": "veh10", "ongoing": false }
  ],
  "vehicles": [
    {
      "id": "veh62",
      "currentLink": "512",
      "currentExitTime": 7201.0,
      "diversionLink": "513",
      "diversionTime": 7211.0,
      "state": "drive",
      "finished": ["pu2"],
      "onboard": ["req2", "req23"]
    }
  ],
  "submitted": [
    {
      "id": "req55",
      "originLink": "5521",
      "destinationLink": "8573",
      "earliestPickupTime": 0.0,
      "latestPickupTime": 7300.0,
      "latestArrivalTime": 8000.0,
      "pickupDuration": 30.0,
      "dropoffDuration": 0.0
    }
  ]
}
```

Implemented

- Request network information to have an idea about the topology

```
{ "@message": "network_request" }
```

```
{
  "@message": "network_response",
  "nodes": {
    "n1": { "x": 50.0, "y": 100.0 },
    "n513": { "x": 5510.0, "y": 106.5 },
  },
  "links": {
    "link1": { "toId": "n1", "toId": "n513", "length_m": 500.0, "freespeed_kmh": 30.0 },
    "link1_back": { "toId": "n513", "toId": "n1", "length_m": 500.0, "freespeed_kmh": 30.0 },
  }
}
```

- Request travel time information from MATSim

```
{ "@message": "travel_time_request", "links": ["l51", "L66"] }
```

```
{ "@message": "travel_time_response", "travelTimes": { "l51": 21.0, "L66": 55.0 } }
```

Future

- Routing service to load off routing information to MATSim
- Charging state, charging stations, ...
- ...

- **Random assignment**
Random request meets random vehicle
- **Bipartite matching**
Minimize overall empty distance
- **Roaming dispatcher**
Cruising and serving next found request
Manual routing using **networkx**
- **Insertion dispatcher**
Python-version of DRT algorithm (almost)
- **Q-learning dispatcher**
Learns a rebalancing matrix over iterations

Chouaki, T., Hörl, S., & Puchinger, J. (2022). Implementing reinforcement learning for on-demand vehicle rebalancing in MATSim. *Procedia Computer Science, The 13th International Conference on Ambient Systems, Networks and Technologies (ANT 2022)*, 201, 134–141. <https://doi.org/10.1016/j.procs.2022.03.020>

```
# As long as we have pending requests or assignable vehicles in
while len(pending_requests) > 0 and len(unassigned_vehicles) >
    # Pop the top request and vehicle
    request = pending_requests.pop(0)
    vehicle_id = unassigned_vehicles.pop(0)

    # Each vehicle that we want to modify needs to get a new se
    stops = []

    # We add one pick-up
    stops.append({
        "link": request["originLink"], # get origin from our sa
        "pickup": [request["id"]], # could add multiple request
    })

    # And the dropoff just after
    stops.append({
        "link": request["destinationLink"], # get destination f
        "dropoff": [request["id"]], # could add multiple reques
    })

    # Note that we do not provide any route between pickup and

    # Put the sequence for this vehicle into the assignment
    assignment["stops"][vehicle_id] = stops
```

- **Random assignment**

Random request meets random vehicle

- **Bipartite matching**

Minimize overall empty distance

- **Roaming dispatcher**

Cruising and serving next found request

Manual routing using **networkx**

- **Insertion dispatcher**

Python-version of DRT algorithm (almost)

- **Q-learning dispatcher**

Learns a rebalancing matrix over iterations

Chouaki, T., Hörl, S., & Puchinger, J. (2022). Implementing reinforcement learning for on-demand vehicle rebalancing in MATSim. *Procedia Computer Science, The 13th International Conference on Ambient Systems, Networks and Technologies (ANT 2022)*, 201, 134–141. <https://doi.org/10.1016/j.procs.2022.03.020>

```
if len(requests) > 0 and len(vehicles) > 0:
    # create location vectors
    request_locations = np.array([r["location"] for r in requests])
    vehicle_locations = np.array([v["location"] for v in vehicles])

    # calculate RxV distance matrix between all requests and vehicles
    distances = np.array([
        la.norm(vehicle_locations - request_location, axis = 1)
        for request_location in request_locations
    ])

    # perform bipartite matching using scipy
    # this gives us pairs of (request_index, vehicle_index)
    # that minimize the overall distance
    matches = [pair for pair in zip(*sopt.linear_sum_assignment(distances))]

    # now construct new sequences for the assigned vehicles
    for request_index, vehicle_index in matches:
        # obtain match
        request = requests[request_index]
        vehicle = vehicles[vehicle_index]
```

- **Random assignment**

Random request meets random vehicle

- **Bipartite matching**

Minimize overall empty distance

- **Roaming dispatcher**

Cruising and serving next found request

Manual routing using **networkx**

- **Insertion dispatcher**

Python-version of DRT algorithm (almost)

- **Q-learning dispatcher**

Learns a rebalancing matrix over iterations

Chouaki, T., Hörl, S., & Puchinger, J. (2022). Implementing reinforcement learning for on-demand vehicle rebalancing in MATSim. *Procedia Computer Science, The 13th International Conference on Ambient Systems, Networks and Technologies (ANT 2022)*, 201, 134–141. <https://doi.org/10.1016/j.procs.2022.03.020>

```
# the network graph for routign
```

```
self.graph = nx.DiGraph()
```

```
for link_id, link_data in data["links"].items():
```

```
    # add an edge with travel time attribute
```

```
    travel_time = 3600 * link_data["length_m"] * 1e-3 / link_data["freespeed_kmh"]
```

```
    self.graph.add_edge(link_data["fromId"], link_data["toId"], travel_time = travel_time)
```

```
def calculate_route(self, origin_link, destination_link):
```

```
    # we need the "to" node of the origin link
```

```
    origin_node = self.edges[origin_link][1]
```

```
    # and the "from" node of the destination link
```

```
    destination_node = self.edges[destination_link][0]
```

```
    # calculate the (quickest) path using networkx
```

```
    path = nx.shortest_path(self.graph, origin_node, destination_node, "travel_time")
```

```
    # now construct the route for the simulator:
```

```
    # - include the origin link
```

```
    # - then add all the intermediary links
```

```
    # - end with the destination link
```

```
    route = [origin_link] + [
```

```
        self.links[edge] for edge in zip(path[:-1], path[1:])
```

```
] + [destination_link]
```

- Road network of Paris provided as an example (OpenStreetMap)
- Fleet generation script: random distribution of vehicles in the network
- Demand generation script
 - Definition of N attractors / emitters with morning / evening factors
 - Requests are generated from / to those points depending on time of day
 - Represents directionality by time of day and spatial imbalance

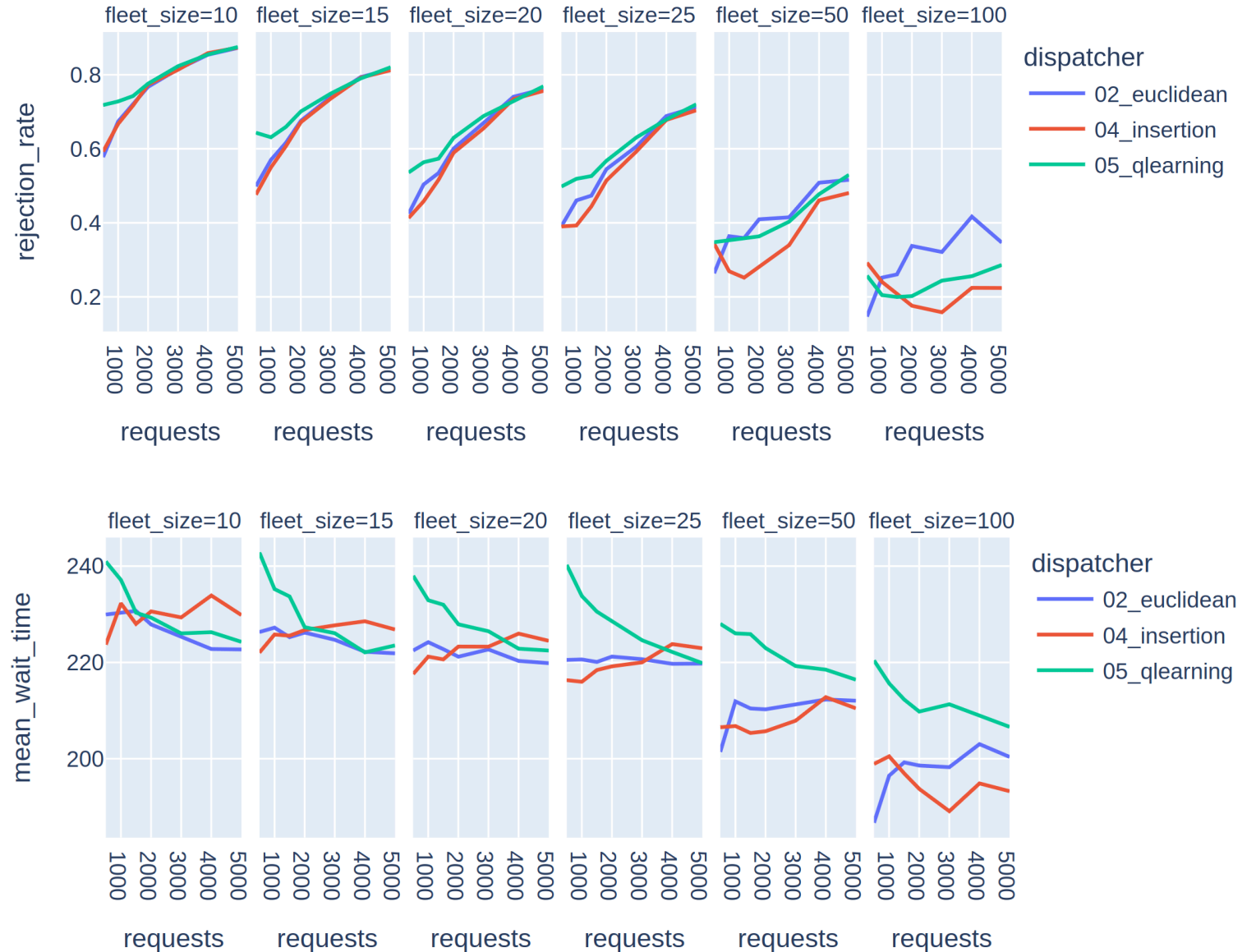
Benchmarking results

Three dispatchers

- Euclidean bipartite matching
- Insertion heuristic (DRT)
- Q-Learning

Analysis

- Average over 10 seeds

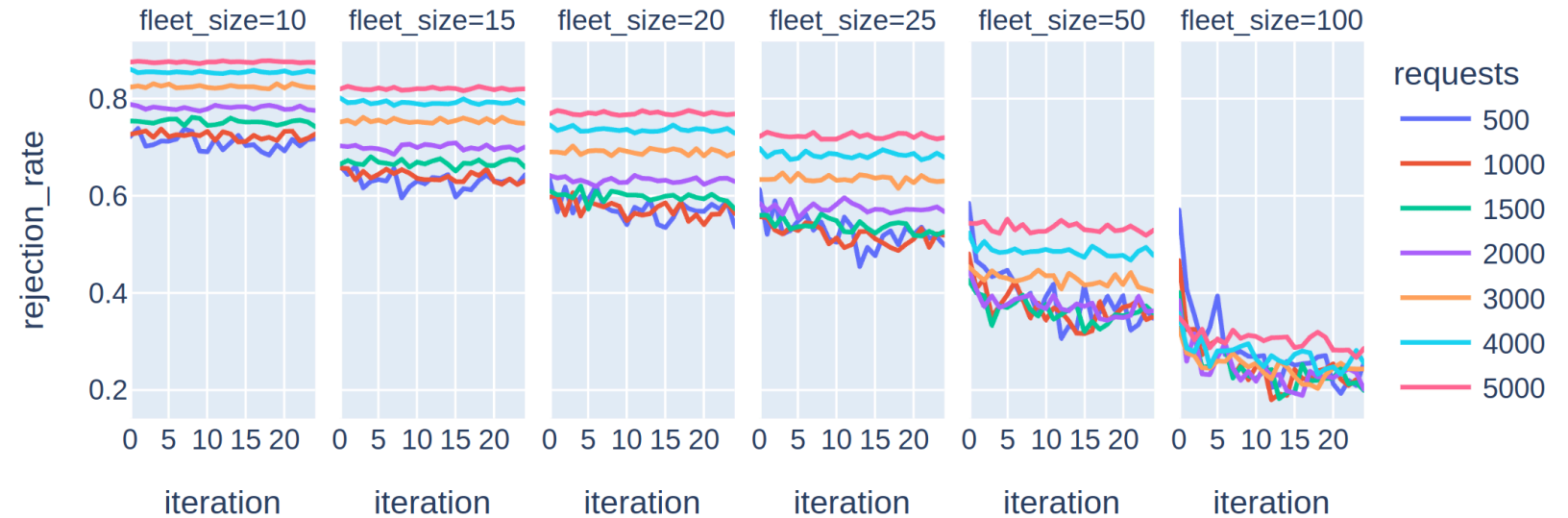


Three dispatchers

- Euclidean bipartite matching
- Insertion heuristic (DRT)
- Q-Learning

Analysis

- Average over 10 seeds
- **Q-Learning**: 25 iterations





As a module

- Replaces individual DRT modes (configurable)
- Port given in configuration or written to **tmp**

```
// create modal parameters
RemoteDrtModeParameters parameters = new RemoteDrtModeParameters();
parameters.setMode(drtConfig.getMode());

// will choose a port randomly
// written to /output/tmp/remote_port_drt
parameters.setPort(0);

// set up global remote config group and add mode parameters
RemoteDrtConfigGroup remoteConfig = new RemoteDrtConfigGroup();
config.addModule(remoteConfig);
remoteConfig.addModeParameters(parameters);

// add functionality to controller
controller.addOverridingModule(new RemoteDrtModule());
```

```
uv run 01_random_dispatcher.py $(cat ../scenario/output/tmp/remote_port_drt)
```



As a module

- Replaces individual DRT modes (configurable)
- Port given in configuration or written to **tmp**

```
// create modal parameters
RemoteDrtModeParameters parameters = new RemoteDrtModeParameters();
parameters.setMode(drtConfig.getMode());

// will choose a port randomly
// written to /output/tmp/remote_port_drt
parameters.setPort(0);

// set up global remote config group and add mode parameters
RemoteDrtConfigGroup remoteConfig = new RemoteDrtConfigGroup();
config.addModule(remoteConfig);
remoteConfig.addModeParameters(parameters);

// add functionality to controller
controller.addOverridingModule(new RemoteDrtModule());
```

Standalone

- Includes parametrizable Paris test case
- Allows to run all benchmarks

```
# generates 1000 requests in scenario/demand.xml
java -cp target/remote-drt-*-SNAPSHOT.jar \
    org.matsim.remote_drt.example.RunGenerateDemand \
    --network-path scenario/paris.xml.zst \
    --output-path scenario/demand.xml \
    --requests 1000
```

1

```
# generates 10 vehicles requests in scenario/fleet.xml
java -cp target/remote-drt-*-SNAPSHOT.jar \
    org.matsim.remote_drt.example.RunGenerateFleet \
    --network-path scenario/paris.xml.zst \
    --output-path scenario/fleet.xml \
    --fleet-size 10 --seats 4
```

2

```
# runs the simulation
java -cp target/remote-drt-*-SNAPSHOT.jar \
    org.matsim.remote_drt.example.RunSimulation \
    --network-path scenario/paris.xml.zst \
    --demand-path scenario/demand.xml \
    --fleet-path scenario/fleet.xml \
    --output-path scenario/output
```

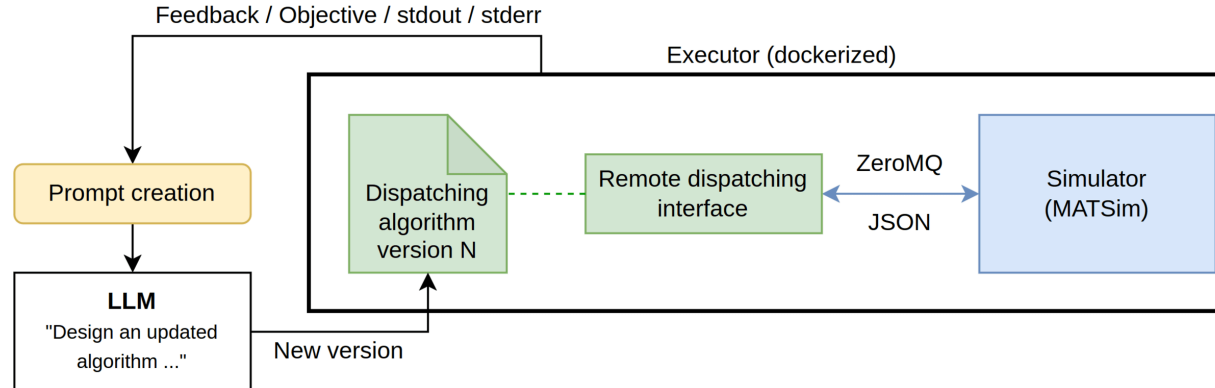
3

```
uv run 01_random_dispatcher.py $(cat ../scenario/output/tmp/remote_port_drt)
```

4

Teaser: LLM-based algorithm adaptation

- Algorithms are often general purpose
- What if we integrate context-specific heuristics (bridges, bottlenecks)?
 - Local knowledge > translation into rules > implementation > testing
 - Work-intensive process!
- Let's let an LLM take an algorithm and customize it iteratively!



Teaser: LLM-based algorithm adaptation

- Algorithms are often general purpose
- What if we integrate context-specific heuristics (bridges, bottlenecks)?
 - Local knowledge > translation into rules > implementation > testing
 - Work-intensive process!
- Let's let an LLM take an algorithm and customize it iteratively!

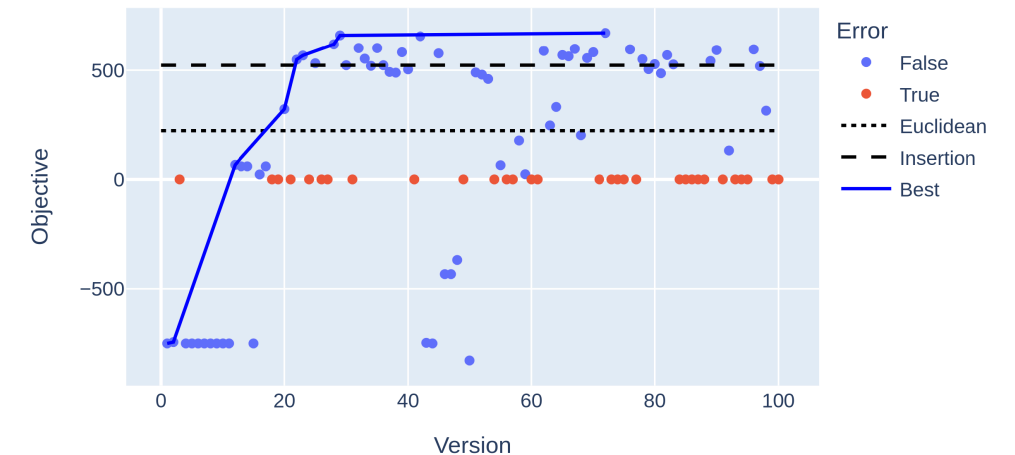
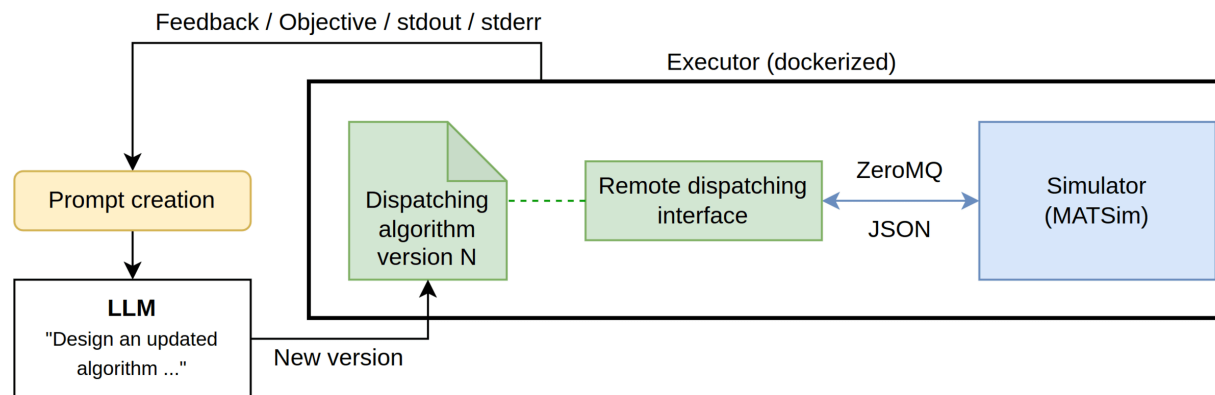


Table 1: Comparison of algorithms

Algorithm	Profit [EUR]	Rides	Profit per ride [EUR]
Euclidean	233	965	0.23
Insertion	523	1061	0.49
Best LLM-based	671	782	0.86

Teaser: LLM-based algorithm adaptation

- Algorithms are often general purpose
- What if we integrate context-specific heuristics (bridges, bottlenecks)?
 - Local knowledge > translation into rules > implementation > testing
 - Work-intensive process!
- Let's let an LLM take an algorithm and customize it iteratively!

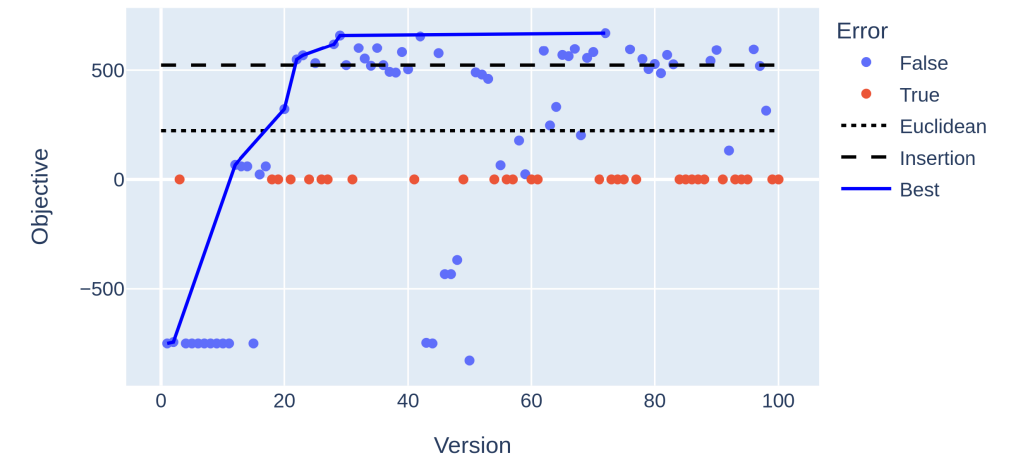
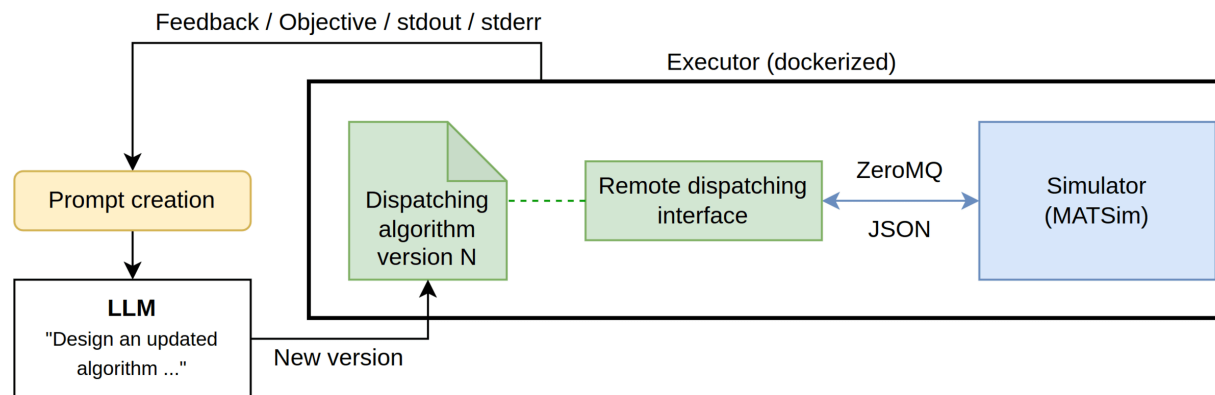


Table 1: Comparison of algorithms

Algorithm	Profit [EUR]	Rides	Profit per ride [EUR]
Euclidean	233	965	0.23
Insertion	523	1061	0.49
Best LLM-based	671	782	0.86

Profit-Optimized Dispatching with Cluster-Specific Profit-Weighted Momentum and Adaptive Rejection Thresholding

Thank you!



Website

<https://sebhoerl.github.io/>



Contact

sebastian.horl@irt-systemx.fr



Icons throughout the presentation: <https://fontawesome.com>